Training Custom ML Models in Python

Build a customer-churn classifier with Pandas, scikit-learn, and Keras; package preprocessing with the model; and understand what custom cloud training adds.

Original study notes based on Gwendolyn Stripling and Michael Abel, First edition, 2023. Book pp. 193–248. Source APIs, services, experiments, and model comparisons reflect the book's publication period. These are explanatory summaries, not verbatim reproductions or a claim that every source detail is retained.

Define churn label → Clean + split → Encode features → Train + evaluate → Package and serve

Detailed Notes

Churn objective and data exploration

Source: book pp. 193–212 / PDF p. 209

Define churn horizon and prediction timestamp. Explore customer tenure, service types, contract/payment categories, charges, and churn prevalence. Resolve missing values and inconsistent types before encoding. Separate identifiers from predictive features and exclude information recorded after the churn outcome.

Use stratified or time-aware splits as appropriate, preserving the test set. Inspect class distributions and category levels in each split. EDA supports decisions about transformations, but should not leak test-set statistics into training.

Feature transformation and logistic regression

Source: book pp. 213–222 / PDF p. 229

One-hot encode nominal categories; scale numeric features where helpful. Fit encoders and scalers on training data only, then reuse the fitted transformations for validation, test, and serving.

$$p(\text{churn}=1|x)=1/(1+e^{-(w\cdot x+b)})$$

Logistic regression models log-odds and gives a probability-like score. Select a threshold based on retention outreach cost and missed churn. A scikit-learn Pipeline keeps preprocessing and estimator together, reducing train/serve mismatch.

Classification metrics and serving predictions

Source: book pp. 223–230 / PDF p. 239

Accuracy is one metric among precision, recall, F1, ROC-AUC, and the confusion matrix. Churn interventions often care about finding likely churners while managing outreach capacity. Evaluate calibration if decisions use predicted probabilities.

Persist the full pipeline, feature schema, category handling, and model version. Validate request inputs and prediction outputs. Monitor actual outcomes once labels mature; the prediction horizon creates a delay before performance is measurable.

Keras neural network and custom Vertex AI training

Source: book pp. 231–248 / PDF p. 247

Keras lets you build a dense classifier with input dimensions matching transformed features, nonlinear hidden layers, and a one-unit sigmoid output for binary classification. Use binary cross-entropy, monitor validation metrics, and guard against overfitting. Compare fairly against logistic regression; greater capacity is not proof of greater value.

Vertex custom training packages code, dependencies, data access, and model output in a managed job. Cloud training helps when scale/resources justify setup, not merely because the service exists. Reuse the same preprocessing code, save the model/pipeline, log parameters, and clean up endpoints. For small datasets, local/Colab experiments may be simpler.

Revision Check

1. What information would leak churn into the feature set?
2. How do you prevent encoding leakage?
3. Which metric matches the intervention capacity?
4. What must be saved with the model for serving?

Custom code gives flexibility, but the full data-to-prediction pipeline must travel with the estimator.

Source links refer to the supplied PDF. Every contents-page topic is mapped in coverage.md. Consult the book for complete implementations, figures, exercises, citations, and experiment evidence.